PYAAS.

ENGINEERING · INTEGRATION FRAMEWORK

Connecting the Stack

How the Pyaas app & backend, Dolibarr and the Rider app integrate into one reliable D2C fulfilment system — the architecture, interfaces, data contracts and build plan for the tech team.

Scope. Three systems — **Pyaas app + backend** (operational system of record), **Dolibarr** (financial & inventory ledger) and the **Pyaas Rider app** (last mile) — plus the supporting services (payments, notifications, chiller telemetry, Milk-Union procurement). This is the integration blueprint, not application-internal design.

FOR
Pyaas Engineering

SYSTEMS
App+Backend · Dolibarr 23.0.3 · Rider app

VERSION
v1.0 · 14 Aug 2026

STATUS
Blueprint — confirm decisions marked in §11

CONFIDENTIAL — Internal engineering document. No credentials or personal data included.

1 Purpose & integration principles

This framework defines **how the three Pyaas systems talk to each other** so that a subscription becomes a delivered, billed, reconciled order every day — at the scale of thousands of subscribers — without any system doing another's job. It is the contract the tech team builds and tests against.

Six principles that govern every interface

Principle	What it means for the build
1. One source of truth per domain	Every data domain (customer, product, stock, order, money) has exactly one owning system. Others hold read-only mirrors, synced — never a second editable copy. (See §2.)
2. App is the engine; Dolibarr is the ledger	The Pyaas backend runs the high-frequency OLTP (subscriptions, daily orders, wallet, POD). Dolibarr receives **aggregated** financial & stock data — never one document per drop.
3. Right pattern per flow	Real-time for payments & notifications; event-driven for delivery/POD; **nightly batch** for order generation and app→Dolibarr posting. Never a synchronous call per drop.
4. Idempotent & retry-safe	Every write carries an external key (`ref_ext` / idempotency key). Re-posting overwrites, never duplicates. All jobs are re-runnable.
5. Reconciliation is the guarantee	A nightly 3-way tie (backend POD = wallet = Dolibarr revenue) proves nothing leaks. It is a first-class service, not an afterthought. (See §9.)
6. Privacy by design (DPDP)	Customer PII stays in the backend. Only the minimum needed crosses to Dolibarr; addresses are never duplicated wholesale. (See §8.)

PRINCIPLE

The integration's job is to keep **the app as the operational brain** and **Dolibarr as the books**, joined by a thin, idempotent, batch-first layer. If a proposed interface makes Dolibarr process per-drop transactions or duplicates the customer master, it violates the framework — redesign it.

2 Master data — single source of truth

Before any interface is built, fix **who owns each data domain**. This table is the most important decision in the whole integration; every sync direction follows from it.

Data domain	System of record	Mirrored to (read-only)	Sync direction & cadence
Customer, address, subscription, pause/skip	PYAAS APP+BACKEND	Dolibarr third-party **only** where a named GST invoice is required	app → Dolibarr, minimal fields, on-demand
Product / SKU / price / GST (HSN)	DOLIBARR	App catalogue	Dolibarr → app, on change / nightly pull
Stock / inventory on-hand	DOLIBARR	App reads “available” for indent	Dolibarr → app (read); app → Dolibarr daily stock movement
Wallet balance / advances	PYAAS APP+BACKEND	Dolibarr liability journal (aggregate)	app → Dolibarr, periodic total
Daily orders / deliveries / POD	PYAAS APP+BACKEND	— (aggregate only)	internal to backend
Revenue / invoices / GST / accounting	DOLIBARR	—	app → Dolibarr daily summary + monthly invoice
Procurement / Milk-Union settlement	DOLIBARR	—	internal to Dolibarr

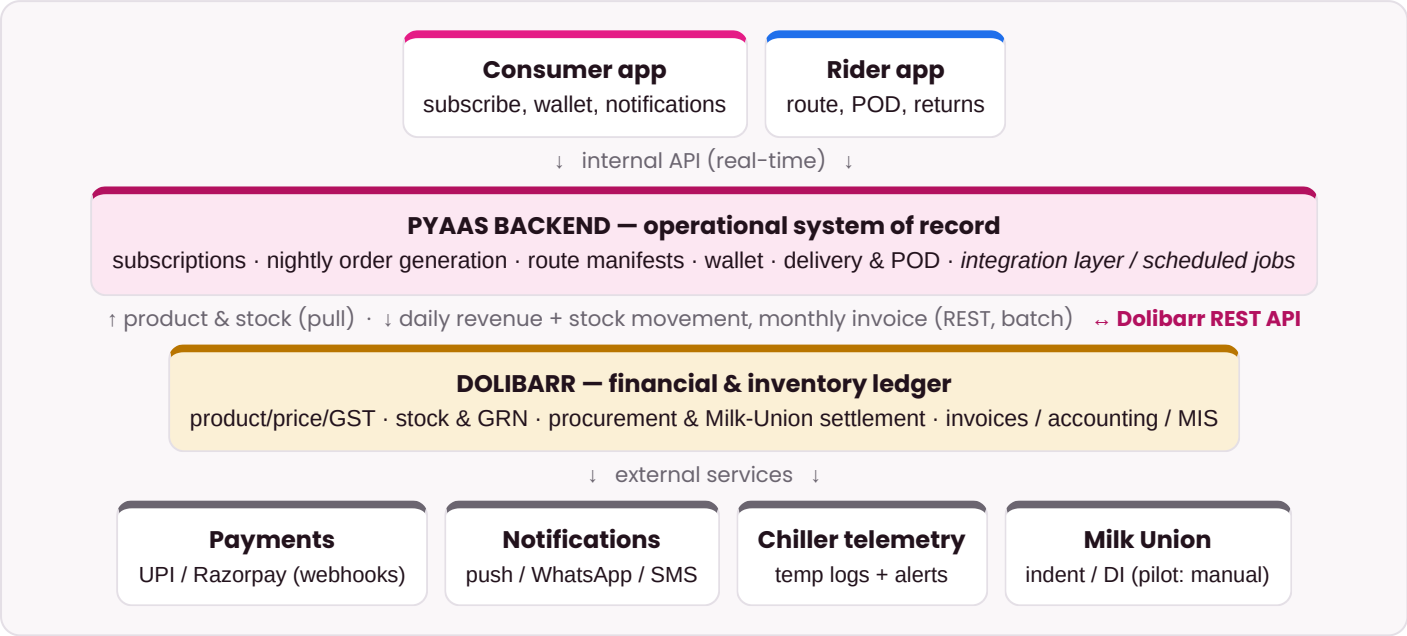
DECISION TO CONFIRM · The Pyaas app currently holds its own product catalogue (e.g. Toned ₹29, Full Cream ₹35), and Dolibarr holds a 63-item product master with buying/selling price. **Pick one master.** Recommended: **Dolibarr owns product/price/GST** (it already carries HSN, cost and TakePOS uses it); the app pulls the catalogue nightly. Confirm and retire the other as editable.

ANTI-PATTERN — DO NOT

Do **not** let two systems both edit the same domain — e.g. prices maintained in both the app and Dolibarr. That guarantees drift. One editable master; everyone else reads a synced mirror keyed by a stable SKU code.

3 Integration architecture & patterns

A **hub-and-ledger** topology: the Pyaas backend is the operational hub that both apps talk to; a thin **integration layer** (scheduled jobs + a few webhooks) connects it to Dolibarr and the external services.



The three integration patterns — and where each is used

Pattern	Used for	Why
Real-time API	Consumer app ↔ backend; Rider app ↔ backend; payment webhooks; delivery notifications	User-facing, must be instant
Event-driven	Delivery captured → POD event → wallet debit → notification; excursion alert	Reacts the moment something happens; decouples producers/consumers
Nightly batch	Order generation (9 PM); app→Dolibarr revenue & stock posting; product/stock pull; monthly invoicing	High volume, off-peak, idempotent — keeps Dolibarr off the hot path

ARCHITECTURE

The **Rider app is a client of the Pyaas backend**, not a separate database. “Integrating the rider app” means exposing two backend interfaces to it: **pull the route manifest** and **push POD/returns events**. It does not talk to Dolibarr at all.

RIDER APP

4 Interface register

Every integration point, with its trigger, pattern and idempotency key. Build and contract-test each one independently. IDs are referenced by the nightly sequence in §5.

ID	Source → target	What	Pattern	Key payload	Idempotency
I1	Consumer app → Backend	Subscription create / modify / pause / skip	Real-time API	subscription, address, qty, slot	backend txn
I2	Backend job (internal)	Nightly order generation @ 21:00 — expand subs → orders, net pause/skip; group by beat; sequence drops	Batch	orders, route trays	job run-id + date
I3	Backend → Rider app	Pull route manifest for the rider's beat	Real-time API	stops, address, seq, items, pay status	manifest-id
I4	Rider app → Backend	Post POD (geo-photo+OTP) & returns per drop	Event / API	order_id, ts, geo, photo, completion, reason	order_id + event
I5	Backend → Consumer app	Out-for-delivery / delivered notification + log	Push / event	order_id, status	order_id + status
I6	Payment PSP ↔ Backend	Wallet top-up, autopay, refunds	Webhook / API	txn_id, amount, status	psp_txn_id
I7	Backend → Dolibarr	Daily revenue summary — 1 line per SKU per day	REST batch ~06:00	date, sku, units, price, amount	`ref_ext=REV-{date}-{sku}`
I8	Backend → Dolibarr	Daily stock movement (dispatched − returned) per SKU	REST batch	date, sku, qty_out, qty_ret	`ref_ext=STK-{date}-{sku}`
I9	Dolibarr → Backend	Product / SKU / price / GST master	Pull on change / nightly	sku, label, price, tax, hsn	sku code
I10	Dolibarr → Backend	Available stock read for the indent	Read pre-cut-off	sku, on_hand	—
I11	Backend → Dolibarr	Monthly subscriber invoice (or Contracts+MBI from app qty)	REST monthly	customer_ref, month lines	`ref_ext=INV-{cust}-{YYYYMM}`
I12	Chiller telemetry → Backend/hub	Temperature logs + excursion alert (≤15 min)	Event / webhook	device, temp, ts, alert	device + ts
I13	Backend/Dolibarr → Milk Union	Indent / DI (prepaid)	Pilot: manual; future API	sku, qty, DI ref	DI ref
I14	Reconciliation service	Nightly 3-way tie (POD = wallet = Dolibarr revenue)	Batch	recon report	date

Interfaces I1–I6 and I12 are within or adjacent to the Pyaas platform (build first — they run the pilot). I7–I11 and I14 are the app↔Dolibarr integration (automate after). PYAAS APP+BACKEND DOLIBARR

5 The canonical nightly data sequence

One end-to-end pass through all systems, with the interface (I#) and owning system for each step. This is the reference the batch jobs and the rider/hub flows must implement.

1. **21:00** PYAAS APP+BACKEND I1
   Order cut-off — backend freezes tomorrow's book; late orders roll to the next day
2. **21:05** PYAAS APP+BACKEND I2, I10
   Order-generation job: expand active subscriptions → concrete orders (net pause/skip), group by beat, sequence drops. Read available stock for the indent
3. **21:30** PYAAS APP+BACKEND I13
   Indent computed and sent to the Milk Union (pilot: manual DI)
4. **~02:00** DOLIBARR —
   Inbound: GRN booked in Dolibarr (stock in) against the dispatch note
5. **02:00–03:15** PYAAS APP+BACKEND I2
   Sortation: backend serves route trays; pick-to-tray scans update tray state
6. **03:30** PYAAS APP+BACKEND —
   Tray scan-out against rider ID (backend records dispatch readiness)
7. **04:00** RIDER APP I3
   Rider app pulls its manifest; dispatch
8. **04:00–07:00** RIDER APP I4, I5, I6
   Deliveries: rider posts POD events → backend debits wallet & fires the customer notification
9. **07:00–07:30** RIDER APP I4
   Returns: undelivered stock scanned back (reason codes); routes closed
10. **~07:45** PYAAS APP+BACKEND I14
    Reconciliation service runs the 3-way tie (POD = wallet = Dolibarr revenue)
11. **~08:00** PYAAS APP+BACKEND I7, I8
    Backend posts the **daily revenue summary** and **stock movement** to Dolibarr (aggregated)
12. **nightly** DOLIBARR I9
    Product / price master pulled from Dolibarr if changed
13. **month-end** DOLIBARR I11
    Monthly subscriber invoices generated in Dolibarr from app-supplied delivered quantity

6 Dolibarr integration specifics (REST API)

The app ↔ Dolibarr link uses Dolibarr's native **REST API** — no connector module needed. It is deployable on your DoliCloud instance today. DOLIBARR

Setup

- Enable the **API/Web services (REST)** module in Dolibarr → Setup → Modules.
- Create a dedicated **integration user** (least privilege: products read; stock write; invoices write) and generate its **API key**.
- Base URL: `https://pyaas.with10.dolicloud.com/api/index.php/`
- Auth header on every call: `DOLAPIKEY: <key>` — store in a secrets manager, never in code.

Endpoints used

Endpoint	Interface	Use
`/products`	I9	Pull SKU / price / tax master
`/warehouses`, `/stockmovements`	I8, I10	Read on-hand; post daily stock movement (PILOT HUB)
`/invoices`	I7, I11	Daily revenue-summary invoice; monthly subscriber invoice
`/thirdparties`	I11	Minimal customer record where a named GST invoice is required
`/contracts`	I11	Optional: standing subscription as a commercial record (with MBI)

Example — post a daily revenue line (idempotent)

```
# One invoice/day to an internal "Daily Subscription Sales" third party, one line per SKU.
POST /api/index.php/invoices   # header: DOLAPIKEY: ***
{
  "socid": 42,                      # internal sales third-party
  "date": "2026-08-15",
  "ref_ext": "REV-2026-08-15",     # idempotency: re-POST updates, never duplicates
  "lines": [
    { "product_id":11, "qty":1190, "subprice":29, "tva_tx":0 },  # Toned
    { "product_id":7,  "qty":770,  "subprice":35, "tva_tx":0 }   # Full Cream
  ]
}
```

ANTI-PATTERN — DO NOT

Do **not** create one invoice per delivery. At thousands of drops that is ~1.8M documents/year and will exhaust DoliCloud. One aggregated revenue invoice per day (lines per SKU) + monthly per-customer invoices only where GST needs them.

DECISION TO CONFIRM · Confirm the **API module is enabled** and create the **integration user + key**. Confirm your **DoliCloud plan's API throughput**; batch all posts off-peak with backoff on 429/5xx. Validate against Dolibarr **23.0.3** object schemas.

7 Key data contracts

The two contracts that “integrate the Rider app” are the **route manifest** (backend → rider) and the **POD event** (rider → backend). Freeze these first — they are the go-live critical path.

ROUTE MANIFEST STOP — I3, BACKEND → RIDER APP

```
{
  "manifest_id": "M-2026-08-15-BEAT7",
  "rider_id": "R-014",  "stop_seq": 12,
  "order_id": "O-9F3A2",  "customer_ref": "C-12345",
  "address": { "society":"Omaxe", "tower":"B", "floor":9, "flat":"B-904",
               "landmark":"main gate", "geo":{"lat":26.85,"lng":80.94} },
  "gate_note": "no calls / bells; leave in milk box",
  "items": [ {"sku":"PRG-TAAZA-500", "qty":1} ],
  "payment": {"mode":"prepaid", "collect":0},
  "flags": {"first_delivery":true, "free_litre":false}
}
```

POD EVENT — I4, RIDER APP → BACKEND

```
{
  "order_id": "O-9F3A2",  "rider_id": "R-014",
  "ts": "2026-08-15T05:12:40+05:30",
  "geo": {"lat":26.8503,"lng":80.9412},
  "completion_type": "delivered_left_at_point",   # or handed_over | guard | not_delivered_*
  "photo_url": "s3://pod/2026-08-15/O-9F3A2.jpg",  "otp_verified": false,
  "items_delivered": [ {"sku":"PRG-TAAZA-500", "qty":1} ],
  "reason_code": null
}
```

NOTE

On a valid **delivered** POD the backend does three things atomically: debit the wallet, fire the customer notification (I5), and mark the order billable for the nightly revenue post (I7). “No POD, no payment” is enforced here — an order with no POD is never billed.

8 Non-functional requirements

Area	Requirement
Auth & secrets	App APIs behind OAuth2 / JWT; Dolibarr via a least-privilege **integration user** + `DOLAPIKEY`. All secrets in a vault, rotated; never in code or the repo.
Privacy (DPDP 2023)	PII (name, address, phone) stays in the backend. Cross only the minimum to Dolibarr; prefer a **customer code** over name+address unless a named GST invoice legally requires it. TLS in transit, encryption at rest, access logs, retention & deletion honoured.
Idempotency & retries	`ref_ext` / idempotency keys on every write; re-run-safe jobs; exponential backoff on 429/5xx; a **dead-letter queue** for failed posts with replay.
Resilience	Dolibarr downtime must **never block the morning**: queue app→Dolibarr posts and replay when it recovers. Circuit-breaker around the REST client.
Observability	Structured logs + an **integration dashboard**: per-job last-success, lag, row counts; alerts on job failure, reconciliation mismatch, and chiller excursion.
Rate & throughput	Batch all Dolibarr posts off-peak (~06:00–08:00), chunked; respect the DoliCloud plan's limits; no per-drop synchronous calls.
Environments	**dev / staging / prod**, with a Dolibarr sandbox for staging; config & keys per environment; feature-flag new interfaces.
Testing	Contract tests per interface (I1–I14); a seeded **reconciliation test**; and a full **dry-run night in staging** before go-live.

9 The reconciliation engine

The nightly reconciliation is what turns three systems into trustworthy books. It is a scheduled service, idempotent and re-runnable, that asserts two things every night.

Dispatched = Delivered (billed) + Returned (not billed) + Failed (credited)

Must equal	Source A (backend)	Source B
Units delivered	count of valid POD events (I4)	= wallet debits = Dolibarr daily revenue units (I7)
Revenue value	Σ (units × price)	= Dolibarr daily revenue-summary value
Stock out	dispatched − returned	= Dolibarr stock movement (I8)

ARCHITECTURE – how it behaves

On a **match**, the day is closed. On a **mismatch**, the service posts what it can, then raises a **named exception** (owner + variance) to the integration dashboard with an SLA to close ≤ 48 h. The reconciliation report is stored per day and is the audit trail — not the POS.

10 Phased build plan & go-live gates

Phase	Scope (interfaces)	Owner	Gate
Phase 0 — Pilot MVP go-live blocker	Backend **order generation** (I2) + **route manifest** (I3) + **Rider-app POD** (I4) + **notifications** (I5) + **payments/wallet** (I6). Dolibarr fed by a **nightly CSV import** (revenue + stock); monthly invoice manual; reconciliation on a sheet.	App/Backend + Rider	Before 15 Aug go-live
Phase 1 — Automate app↔Dolibarr	**I7** daily revenue summary + **I8** stock movement via REST; **I9** product/price pull; `ref_ext` idempotency; job monitor + alerts.	Backend + Integration	~D15–D30
Phase 2 — Finance & scale	**I11** automated monthly invoicing (Contracts + MBI, or backend billing); **I14** automated reconciliation dashboard; **I12** telemetry alerts; DLQ / backoff hardening.	Integration + Finance	~D30–D55

ARCHITECTURE – what actually blocks go-live

Phase 0 is the **only** part that must exist for the pilot to run at all — and none of it is a Dolibarr feature; it is the app backend + rider app. If Phase 0 is not ready, the hub cannot pack and the rider has no address. Everything Dolibarr-facing (I7–I11, I14) can start as a nightly CSV and be automated after go-live.

11 Risks, anti-patterns & decisions to confirm

Anti-patterns — the framework forbids these

- One Dolibarr document (order/invoice/POS sale) **per delivery** — use daily aggregates.
- Duplicating the full **customer master + addresses** into Dolibarr — keep PII in the backend.
- **Two editable masters** for the same domain (e.g. price in both app and Dolibarr) — one owner, others mirror.
- **Synchronous per-drop** calls to Dolibarr, especially in the 9 PM / 4 AM spikes — batch off-peak.
- Letting Dolibarr downtime **block the morning** — queue & replay.

Top risks

Risk	Mitigation
DoliCloud API throughput / downtime at scale	Batch + backoff + queue/replay; consider a dedicated, tuned instance before high volume
Rider app not shipped / no manifest surface	Phase 0 is the go-live gate; treat I2–I4 as the critical path
Product-master duality (app vs Dolibarr)	Pick one master (recommend Dolibarr); sync the other read-only
PII spread across systems	Customer code over name+address; DPDP data-minimisation review

Decisions to confirm before build

DECISION TO CONFIRM · Product/price/GST master = Dolibarr (recommended) or the app? Retire the other as editable.

DECISION TO CONFIRM · The SOP's **"hub app"** = the operator view of the Pyaas app (not a 4th system)? Confirm the screen that generates route trays and scan-out.

DECISION TO CONFIRM · Are **monthly per-customer GST invoices** legally required (milk is often nil-rated)? If not, a daily revenue summary + accounting may suffice — simpler.

DECISION TO CONFIRM · Payment PSP (UPI provider / Razorpay) and **notification provider** (push / WhatsApp e.g. Interakt / SMS) — confirm which, for I6 and I5.

DECISION TO CONFIRM · Enable the Dolibarr **REST API module**, create the **integration user + key**, and confirm the **DoliCloud plan** throughput.

In one line: keep the **app** as the operational brain and the **rider app** as its last-mile client; make **Dolibarr** the books; join them with a **thin, idempotent, batch-first** integration layer and a **nightly reconciliation** — and the whole stack behaves as one system that scales.

PYAAS Dairy Private Limited · Tech Integration Framework v1.0 · 14 Aug 2026. Blueprint for engineering; confirm the decisions in §11 before build. No credentials or personal data included.